

Origin Safety Intelligence Engine — Architecture Package

Document number: ORIGIN-SIE-ARCH-001 **Revision:** A (initial architecture) **Date:** 2026-09-06 **Owner:** Chris Hawkins, Origin Management Solutions **Status:** For review — no application code is written until this package is approved.

0. Read this first (plain-language summary)

The Origin Safety Intelligence Engine (SIE) is **not a chatbot**. It is a rules-and-knowledge system that reaches OSHA and contractor-compliance conclusions the same way a senior safety professional would: by looking up the actual regulation, applying a fixed set of decision rules, and citing the source. A language model is used **only to explain those conclusions in readable English** — it never decides what the rule is, and the whole system keeps working with every external AI service switched off.

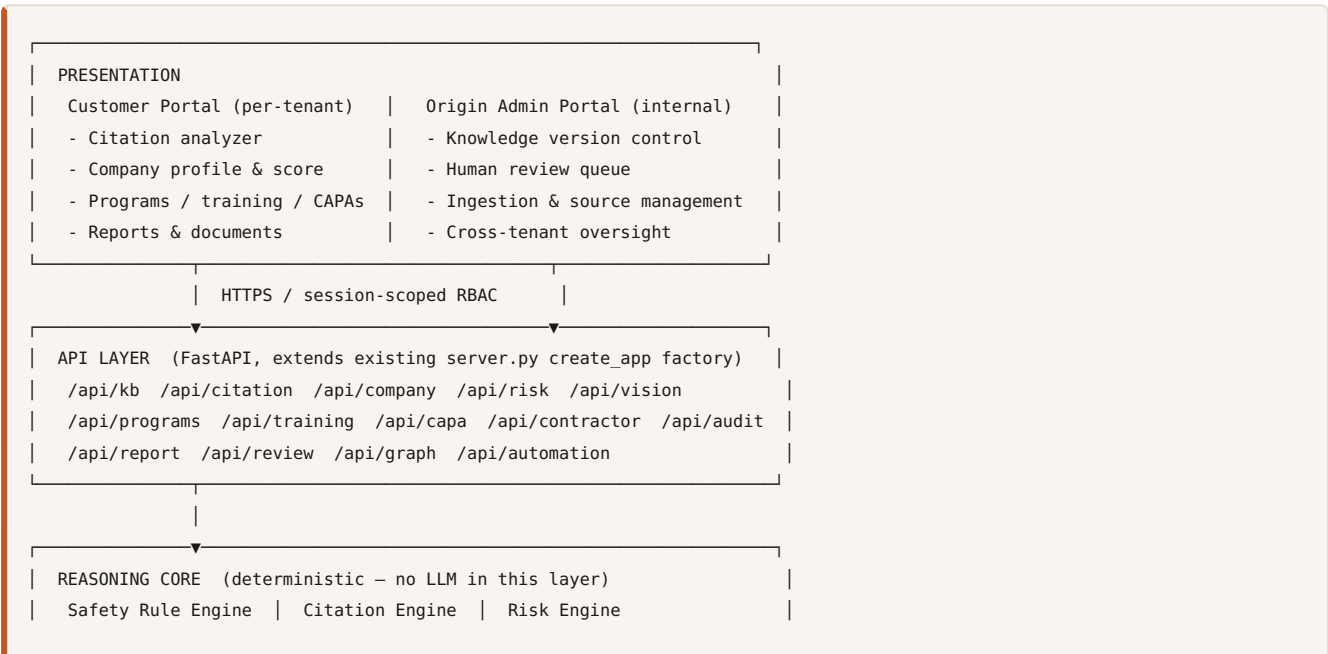
Three ideas hold the whole design together:

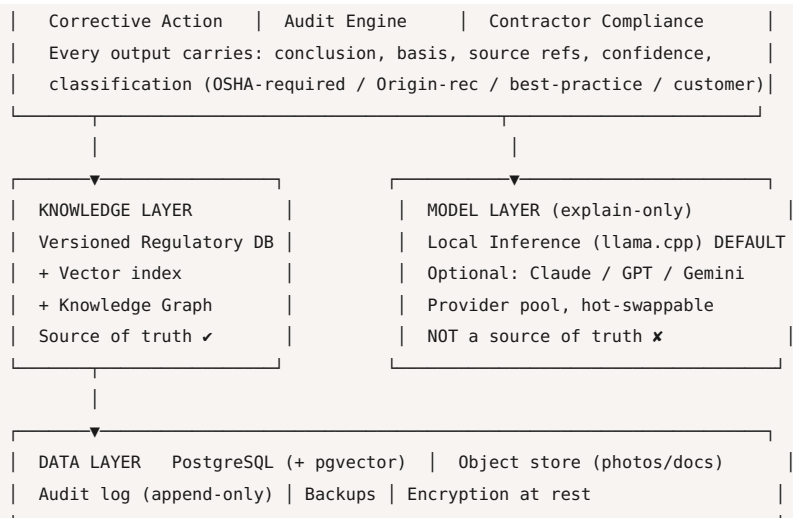
1. **Source of truth is the knowledge base, not the model.** Every regulatory statement is traceable to a stored OSHA/consensus/customer record with a citation and a source URL. If we can't trace it, the system says *"Unable to verify this requirement from the current knowledge base."*
2. **A deterministic rule engine drives every conclusion.** Industry → NAICS → tasks → hazards → standards → programs → training → inspections → documentation → corrective actions. The same inputs always produce the same outputs. No randomness in the compliance path.
3. **The model is a swappable explainer.** It runs locally (llama.cpp) by default. External models (Claude/GPT/Gemini) are *optional accelerators* selected through the existing provider pool. Pull every API key and the platform still analyzes citations, scores companies, generates programs, and produces reports.

We are **not building this from a blank page**. The existing Origin FastAPI app already implements most of the 15 engines in a lighter form. This package tells us what to keep, what to formalize, and what to add.

1. Complete system architecture

1.1 Layered view





1.2 The one rule that makes it trustworthy

Data flows **up** for facts and **down** for language:

- A conclusion is *computed* in the Reasoning Core from Knowledge Layer records. That gives us the answer, the citation, and the confidence.
- The Model Layer is then handed the finished conclusion and asked only to *phrase it*. It is never asked "what does OSHA require?" — it is asked "write this verified finding in plain English."

Because of this split, a wrong, retired, or offline model can degrade the *wording* but can never corrupt the *compliance answer*.

1.3 The 15 engines mapped to existing code

#	Engine (spec)	Status today	Module(s) to build on
1	Regulatory Knowledge Engine	Exists	<code>compliance_kb.py</code> (records, verbatim, osha_index, prequal, abatement)
2	Safety Rule Engine	Exists (partial)	<code>scoping.py</code> (<code>triggered_standards</code>), <code>gaps.py</code> , <code>sector_content.py</code>
3	Document Intelligence Engine	Exists	<code>gaps.py</code> (<code>extract_text</code> , <code>parse_deficiency_report</code>)
4	Computer Vision Engine	Exists	<code>photo_audit.py</code> (self-healing vision chain, KB-cited)
5	Company Risk Engine	Exists (partial)	<code>contractors.py</code> (<code>_risk</code>), <code>compliance_grading.py</code> (<code>estimate_grade</code>)
6	OSHA Citation Engine	Exists (partial)	<code>citation_guard.py</code> , <code>compliance_kb.by_citation</code> , <code>abatement.py</code>
7	Training Engine	Exists (partial)	<code>compliance_kb.training_requirement</code> , <code>gaps.draft_programs</code>
8	Corrective Action Engine	Exists	<code>abatement.py</code> (Draft → Approved → Uploaded → Verified ladder)
9	Safety Program Generator	Exists	<code>gaps.draft_programs</code> , <code>program_docx_bytes</code> , <code>sector_content.py</code>
10	Contractor Compliance Engine	Exists	<code>contractors.py</code> , <code>compliance_grading.py</code> , <code>platform_sub.py</code>
11	Audit Engine	Exists (partial)	<code>gaps.find_gaps</code>
12	Report Generator	Exists (partial)	<code>gaps.program_docx_bytes</code> , <code>executive.py</code>
13	Knowledge Graph	New	<code>new_graph.py</code> over the versioned DB
14	Local Model Inference Layer	Exists	<code>llm/llamacpp_provider.py</code> + provider pool
15	Human Review / Audit Layer	New (partial)	extend <code>citation_guard.py</code> + new <code>review.py</code>

Takeaway: 12 of 15 engines already have working code. The architecture's real job is (a) put them all on one **versioned, traceable knowledge base**, (b) formalize the **deterministic rule engine** they share, and (c) add the **knowledge graph, review queue, and multi-tenant persistence** the spec requires.

2. Database schema

PostgreSQL, extending the existing `platform_db.py` SQLAlchemy models (which already give us `Tenant`, `User`, `Subcontractor`, `ComplianceStatus`, `COI`, `Document`, `ActivityLog` with session-slug tenant isolation). New tables below; every business table carries `tenant_id`, `created_at`, `updated_at`, `created_by`.

2.1 Knowledge & sourcing (the source of truth)

sources — where a fact came from. `id`, `source_type` (`regulation|directive|loi|technical_manual|publication|state_plan|consensus_standard|origin_proprietary|customer_requirement`), `title`, `publisher` (`OSHA|state|ANSI|NFPA|customer|origin`), `url`, `jurisdiction`, `retrieved_at`, `license_note`, `is_copyright_restricted` (bool)

knowledge_versions — every ingest is a version, nothing is overwritten. `id`, `source_id`, `version_label`, `effective_date`, `superseded_by` (fk self, nullable), `ingested_at`, `ingested_by`, `checksum`, `is_current` (bool)

regulations — the atomic regulatory records. `id`, `source_id`, `knowledge_version_id`, `regulation_number` (e.g. 1926.501), `section`, `paragraph`, `title`, `body_text`, `jurisdiction`, `industry_scope[]`, `hazard_category[]`, `applicability_rule` (jsonb), `effective_date`, `superseded` (bool), `confidence`, `embedding_vector`(768)

standards — consensus standards (ANSI/NFPA/etc.), same shape, `copyright_restricted` respected (store citation + summary, not full copyrighted body — see NFPA 70E rule in memory).

kb_embeddings — pgvector index rows for semantic retrieval (or embedding stored inline as above). Retrieval always returns the record id so the citation and version travel with the match.

2.2 Company & compliance

companies — the Company Safety Profile. `id`, `tenant_id`, `legal_name`, `dba`, `naics_code`, `sic_code`, `industry`, `state`, `headcount`, `task_list[]`, `hazard_exposure[]`, `emr`, `trir`, `dart`, `ltir`, `srte`, `fatalities_5yr`, `isn_status`, `avetta_status`, `ravs_status`, `pec_status`, `veriforce_status`, `gc_relationships[]`, `insurance_summary` (jsonb), `known_deficiencies[]`, `safety_score` (int), `score_explanation` (jsonb), `score_updated_at`

employees `id`, `company_id`, `name`, `role`, `hire_date`, `certifications` (jsonb) **projects** `id`, `company_id`, `name`, `site`, `scope`, `operator`, `start_date`, `end_date`, `status` **equipment** `id`, `company_id`, `type`, `model`, `inspection_due`, `cert_status`

2.3 Hazards, findings, citations, actions

hazards `id`, `name`, `category`, `typical_standards[]`, `severity_default` **inspections** `id`, `company_id`, `inspection_number`, `source` (`OSHA|state|internal`), `date`, `inspector`, `status` **citations** `id`, `company_id`, `inspection_id`, `citation_number`, `standard_cited`, `paragraph`, `description`, `classification` (`other|serious|willful|repeat|failure_to_abate`), `initial_penalty`, `current_penalty`, `abatement_date`, `gravity`, `source_ref` **findings** — output of vision/audit/document analysis. `id`, `company_id`, `origin` (`vision|audit|document|manual`), `object_detected`, `potential_hazard`, `location`, `confidence`, `possible_standard`, `human_verification_required` (bool), `classification`, `source_refs[]`, `status` **corrective_actions** — CAPA with full trail. `id`, `finding_id|citation_id`, `description`, `type` (`immediate|recommended`), `owner`, `due_date`, `status` (`Draft-Approved-Uploaded-Verified-Closed`), `evidence[]`, `root_cause`, `recurrence_prevention`, `closed_at`, `closed_by` (reuses `abatement.py` ladder) **incidents** `id`, `company_id`, `date`, `type`, `recordable` (bool), `recordability_basis` (jsonb from `recordability.py`), `dart_days`

2.4 Programs, training, audits, documents

programs `id`, `company_id`, `program_type`, `title`, `body_markdown`, `source_refs[]`, `version`, `generated_from` (jsonb rule trace), `status` **training** `id`, `company_id`, `topic`, `required_by_standard`, `frequency`, `matrix` (jsonb), `toolbox_talk_id`

training_records id, employee_id, training_id, completed_at, certificate_ref, refresher_due **audits** id, company_id, type, scope, findings[], score, conducted_at **documents** id, company_id, doc_type, storage_ref, sha256, generated_by, version **photos** id, company_id, storage_ref, taken_at, findings[]

2.5 Customer / contractor requirements

customers — the operator/GC/owner/insurer a contractor must satisfy.
id, tenant_id, name, type (isn|avetta|pec|ravs|veriforce|gc|owner|insurer) **customer_requirements** id, customer_id, requirement_text, mapped_standard (nullable), is_stricter_than_oshra (bool), source_ref — **stored per-customer, never merged into OSHA records.**

2.6 Reports, review, audit, users

reports id, company_id, report_type, document_number, revision, body_ref, source_references[], approval_status, approved_by, revision_history (jsonb) **approvals** id, entity_type, entity_id, requested_by, approver, decision, decided_at, notes **review_queue** — Human Review Layer. id, entity_type, entity_id, reason (legal|contested|ambiguous|injury|fatality|criminal|pe_judgment|low_confidence), status, assigned_to, resolved_by, resolution **audit_log** — append-only, already partially in ActivityLog. id, tenant_id, actor, action, entity_type, entity_id, before (jsonb), after (jsonb), at, ip **users** extends existing User: role (super_admin|origin_staff|customer_admin|customer_user|read_only), tenant_id, mfa_enabled

2.7 Referential integrity that enforces traceability

A NOT NULL constraint ties every regulatory conclusion back to a record: - findings.source_refs and citations.source_ref must reference real regulations / standards / customer_requirements ids. - programs.source_refs and reports.source_references likewise. - Nothing that makes a regulatory claim can be saved without a source id. This is the database enforcing "*never fabricate.*"

3. Knowledge architecture

3.1 Corpus and ingestion

The existing single-corpus approach (corpus.jsonl powering chat + GapFinder + docs, per memory) is generalized into the versioned sources / knowledge_versions / regulations / standards tables. Ingestion pipeline (ingest.py , new):

1. **Fetch** from an approved source (OSHA eCFR, directives, LOIs, Technical Manual, state-plan pages, licensed consensus standards, Origin proprietary docs, customer requirement uploads).
2. **Parse** into atomic records (one paragraph/section = one row), each stamped with source, section, regulation number, effective date, jurisdiction, industry, hazard category, applicability rule, source URL, retrieval date.
3. **Version** — create a knowledge_versions row; if it supersedes an existing regulation, set the old row's superseded=true and link superseded_by . **Old versions are never deleted** (needed to explain a citation written under a prior rule).
4. **Embed** — compute a local embedding (sentence-transformers, runs offline) → vector(768) for semantic search. Keyword/verbatim index (already in compliance_kb.verbatim_search) is kept as the exact-match path.
5. **Confidence** — assign a retrieval confidence (exact citation match = high; semantic-only = medium; inferred applicability = flagged).

3.2 Copyright discipline

Consensus standards (NFPA 70E and similar) are is_copyright_restricted=true : store the citation, effective date, and an Origin-authored summary — never the full copyrighted text. The retrieval layer refuses to emit restricted bodies verbatim.

3.3 Retrieval contract

Every retrieval returns a list of **records**, not prose: {regulation_id, regulation_number, section, body_text, source_url, effective_date, jurisdiction, superseded, confidence} . Exact-citation lookups (by_citation , verbatim_text) win over semantic

matches. If nothing clears the confidence floor, retrieval returns empty and the caller emits *"Unable to verify this requirement from the current knowledge base."*

3.4 Knowledge Graph (engine 13, new `graph.py`)

A directed graph materialized from the relational tables (adjacency rows + optional pgvector-backed traversal), expressing:

```
Company -industry→ NAICS -exposes→ Hazard -governed_by→ Standard
Standard -requires→ Program -and→ Training -documented_by→ Document
Finding -triggers→ CorrectiveAction -proven_by→ Evidence -achieves→ Closure
Customer -imposes→ Requirement -(maps_to|stricter_than)→ Standard
Company -has_history_with→ Customer / OSHA
```

The graph is what powers "show me *why* this company needs this program" — every edge points at a source record, so an explanation is just a walk of sourced edges.

4. Reasoning architecture (deterministic core)

4.1 The decision pipeline

All regulatory conclusions flow through one ordered, pure-function pipeline (formalized from today's `scoping.py` + `gaps.py`):

```
industry → naics_map → task inventory → hazard set
  → triggered standards (scoping.triggered_standards)
  → required programs (gaps.required_set)
  → required training (compliance_kb.training_requirement)
  → inspection/recordkeeping obligations (scoping._recordkeeping_obligation)
  → documentation requirements
  → corrective actions (on findings/citations)
  → ranked recommendations
```

Each stage is a deterministic function: same inputs → same outputs, no model call, fully unit-testable. Each emitted item carries a **basis** (the rule that fired) and **source_refs** (the regulation ids).

4.2 Output envelope (every conclusion looks like this)

```
{
  "conclusion": "Fall protection training is required for this crew.",
  "classification": "OSHA-required",
  "basis": "Residential construction (NAICS 236115) + work >6 ft triggers 1926.503",
  "source_refs": ["reg:1926.503", "reg:1926.501(b)(13)"],
  "confidence": 0.94,
  "human_review_required": false,
  "explanation_text": "<<filled in by model layer, phrasing only>>"
}
```

4.3 The four-way classification (never mixed)

Every statement is tagged exactly one of: - **OSHA-required** — traceable to a current CFR/state-plan record. - **Origin recommendation** — Origin proprietary best practice, sourced to an Origin record. - **Best practice** — industry consensus, not legally mandated. - **Customer requirement** — from `customer_requirements`, stored per customer, may be stricter than OSHA.

The UI renders these in four visually distinct bands. The rule engine will not emit a single item that blends categories.

4.4 Explainability ("WHY?")

Because the basis and source_refs travel with every conclusion, the "WHY?" button is not an LLM prompt — it renders the actual rule trace and links each source record. No black box.

5. Model architecture (explain-only, swappable, offline-capable)

5.1 Provider pool (exists)

The current `Engine.pool` already abstracts providers behind `.name/.model/.client` with `has()/provider()/names()`. Providers: `anthropic`, `openai`, `grok`, `gemini`, `ollama`, `llamacpp` (**local, self-contained**).

5.2 Default = local

Production default provider becomes `llamacpp` (llama.cpp, ~1.9GB Q4_K_M, already implemented in `llm/llamacpp_provider.py`). External providers are **optional accelerators**, selected per task through config — exactly the pattern already proven in the Photo Audit self-healing vision chain (`vision.order`, `gemini-flash-latest` alias, auto-fallback).

5.3 The hard constraint, enforced

- The Reasoning Core and Knowledge Layer make **zero** model calls.
- The model is invoked only by a thin `explain(conclusion) -> text` service and by optional vision description.
- A config flag `require_local_only: true` forces the pool to `llamacpp` and refuses external calls.
- **Test:** with `ANTHROPIC_API_KEY`, `OPENAI_API_KEY`, `GEMINI_API_KEY`, `XAI_API_KEY` all unset, the platform must still: analyze a citation, score a company, generate a program, run an audit, and produce a report. Only the *prose polish* falls back to the local model. This is a required CI test (\$10).

5.4 Swappability

Upgrading the local model = drop a new GGUF and change one config line. No engine, schema, or API change. The model is never the source of truth, so a swap can change tone but not compliance answers.

6. Safety-rule architecture

6.1 Rule representation

Rules are **data, not code** where possible — stored as `applicability_rule` (jsonb) on regulations plus a compiled rule table, so Origin staff can add/adjust triggers without a deploy. A rule row:

```
{
  "id": "rule_fall_res_6ft",
  "when": {"naics_prefix": ["2361", "2362"], "task": ["roofing", "framing"], "height_ft_gt": 6},
  "then": {"standards": ["1926.501(b)(13)", "1926.503"],
    "programs": ["fall_protection"], "training": ["fall_protection"],
    "classification": "OSHA-required"},
  "source_refs": ["reg:1926.501(b)(13)"]
}
```

6.2 Evaluation

The Safety Rule Engine loads active rules, evaluates each against a Company Safety Profile deterministically, and unions the results into the decision pipeline (§4.1). Sector-specific content is injected the way `sector_content.py` already does per-NAICS (avoiding hand-writing 600 documents). Conflicts (customer stricter than OSHA) are surfaced side-by-side, never silently merged.

6.3 Versioning

Rules are versioned with the knowledge base. A citation written in 2023 is explained against the rule/regulation version effective on its date, using `knowledge_versions.effective_date` — which is why old versions are retained.

7. API architecture

FastAPI, extending the existing `create_app(config, engine, token)` factory (no module-level app). All routes tenant-scoped and RBAC-gated. Representative endpoints per engine:

Knowledge	GET /api/kb/search?q=	POST /api/kb/ingest (admin)
	GET /api/kb/regulation/{id}	GET /api/kb/versions/{source_id}
Citation	POST /api/citation/analyze	(upload citation/PDF/photos/response)
	GET /api/citation/{id}	-> 12-part output object
Company	POST /api/company	GET/PUT /api/company/{id}
Risk	GET /api/risk/{company_id}	-> score + explanation
Vision	POST /api/vision/analyze	(photos -> findings, review-flagged)
Programs	POST /api/programs/generate	GET /api/programs/{id}
Training	POST /api/training/matrix	POST /api/training/toolbox
CAPA	POST /api/capa	PUT /api/capa/{id}/status
Contractor	GET /api/contractor/{id}	GET /api/contractor/{id}/customers
Audit	POST /api/audit/run	GET /api/audit/{id}
Report	POST /api/report/generate	GET /api/report/{id}
Review	GET /api/review/queue	PUT /api/review/{id}/resolve
Graph	GET /api/graph/company/{id}	GET /api/graph/why/{finding_id}
Automation	GET /api/automation/alerts	POST /api/automation/scan

Citation analyzer response (engine 6) returns all 12 required outputs in one object: `summary`, `source`, `hazard_explanation`, `immediate_corrective_action`, `recommended_corrective_action`, `root_cause`, `training`, `documentation`, `evidence_checklist`, `followup_checklist`, `recurrence_prevention`, `abatement_tracking` — each item classified per §4.3 and carrying `source_refs`.

Every response includes the standard envelope fields (`confidence` , `human_review_required` , `source_refs`) so the frontend can render disclaimers and review flags uniformly.

8. UI architecture

Two portals over the same API, RBAC-separated.

Customer Portal (per tenant): - Dashboard: company safety score + explanation, open findings, expiring training, open CAPAs, upcoming abatement deadlines. - Citation Analyzer: upload → 12-part result, four-color classification bands, "WHY?" trace, evidence checklist. - Vision Inspector: photo upload → findings labeled "*Potential hazard detected — human verification required*," never "*violation confirmed*." - Programs / Training / CAPA / Reports: generate, track, download Origin-branded documents. - Contractor Compliance board: ISN/Avetta/RAVS/PEC/Veriforce/GC status (reuses `contractors.py` status dots).

Origin Admin Portal (internal): - Knowledge version control: sources, versions, supersession, confidence tuning. - Human Review queue: legal/contested/ambiguous/injury/fatality/criminal/PE-judgment/low-confidence items. - Ingestion & source management; rule editor (jsonb rules). - Cross-tenant oversight, audit-log viewer.

Design language reuses the existing Origin portal (session-slug tenanting, orange accent). Every AI-derived statement shows a confidence chip and, where flagged, a review banner. Disclaimer footer on regulatory outputs: not attorney / not OSHA official / not PE / not a licensed safety professional.

9. Security architecture

Builds on the hardening already in place (per memory): auto-generated secret, admin-fails-closed, PBKDF2 PINs, lockout, session-slug tenant isolation.

- **RBAC:** five roles (`super_admin`, `origin_staff`, `customer_admin`, `customer_user`, `read_only`); every endpoint checks role + tenant.
- **Tenant separation:** every business row carries `tenant_id` ; queries are tenant-scoped at the session layer — no cross-account bleed. Customer requirements and profiles are stored independently per customer.
- **Encryption:** TLS in transit; PostgreSQL encryption at rest; object store (photos/docs) encrypted; secrets from host env only (`ORIGIN_ADMIN_PASSWORD` , DB URL, optional API keys).
- **Append-only audit log:** every create/update/approval/review with before/after, actor, timestamp, IP.
- **Versioning & backup:** knowledge and rules versioned; nightly encrypted DB backups; object-store versioning.
- **Human-in-the-loop gates:** high-stakes categories cannot be auto-published; they route to `review_queue` first.
- **Injection resistance:** uploaded documents and photos are treated as data, never instructions; the model layer never receives tool authority over the reasoning core.

10. Implementation roadmap

Each phase ends with a **CI gate: the whole phase must pass its tests with all external LLM keys unset**.

Phase 1 — Core DB + regulatory knowledge + citation analyzer Stand up PostgreSQL schema (§2.1–2.3), migrate `platform_db.py` . Build `ingest.py` + versioned KB from existing `compliance_kb` corpus. Ship Citation Engine (12-part output) reusing `citation_guard.py` + `by_citation` + `abatement.py` . Gate: analyze a real citation offline, fully sourced.

Phase 2 — Company profiles + risk + corrective action `companies` profile + dynamic safety score (formalize `compliance_grading.estimate_grade` + `contractors._risk` with written explanation). CAPA tracker on the Draft → Verified → Closed ladder. Gate: score a company and open/close a CAPA with audit trail.

Phase 3 — Safety program generator + training + JHA/JSA Rule-driven program/training generation (`gaps.draft_programs` , `sector_content.py` , `compliance_jha.py`), Origin-branded `.docx` . Gate: generate an industry-specific program with source refs.

Phase 4 — Computer vision Formalize `photo_audit.py` into the Vision Engine writing `findings` rows, review-flagged, KB-cited, self-healing provider chain already built. Gate: photo → sourced potential-hazard findings, offline-capable via local vision fallback.

Phase 5 — Customer/contractor compliance (ISN/Avetta/RAVS/PEC/Veriforce) `customers` + `customer_requirements` stored per customer; contractor compliance board; grading per platform (`compliance_grading._weights_for`). Gate: two customers with conflicting requirements shown side-by-side, unmixed.

Phase 6 — Autonomous monitoring `automation` scanner: detect new citations, approaching deadlines, expiring training/certs, open CAPAs, repeated hazards, new requirements/sources, risk increases → create tasks/alerts. Gate: seeded events produce correct alerts with no manual trigger.

Appendix A — Constraint compliance checklist

Requirement	How the architecture satisfies it
Not a chatbot	Reasoning Core is deterministic; model only phrases finished conclusions

Requirement	How the architecture satisfies it
No external-LLM dependency	Default <code>llamacpp</code> ; <code>require_local_only</code> flag; CI test with all keys unset
Model swappable, not source of truth	Provider pool; drop-in GGUF; zero model calls in reasoning/knowledge layers
Source of truth = OSHA/consensus/Origin/customer	Versioned <code>sources</code> / <code>regulations</code> / <code>standards</code> ; NOT NULL <code>source_refs</code>
Never fabricate	DB enforces <code>source_refs</code> ; empty retrieval → "Unable to verify..."
Traceable answers	Output envelope carries basis + <code>source_refs</code> + confidence
Deterministic rule engine	Pure-function pipeline §4.1; data-driven rules §6
Versioned knowledge	<code>knowledge_versions</code> , supersession, retained history
OSHA-req vs Origin-rec vs best-practice vs customer never mixed	Four-way classification §4.3, four UI bands
Vision says "potential hazard," not "violation"	Vision Engine wording rule §8, <code>human_verification_required</code> flag
Confidence + WHY + human review	Envelope confidence, rule-trace WHY, <code>review_queue</code> gates
PostgreSQL + vector + graph	<code>pgvector</code> embeddings + <code>graph.py</code>
RBAC, tenant separation, encryption, audit, backup	§9
Customer + admin portals, automation	§8, Phase 6

End of architecture package. On approval, implementation begins at Phase 1.